Name: Durga Solanke Class:Sy B Roll No:47

Practical no 1

Aim: Use sequence data types and their associated operations in Python a. Strings b. Lists c. Arrays d. Tuples e. Dictionary f. Sets g. Range

1. What are sequence data types in Python? Name them and explain how indexing and slicing work across different sequence types Sequence data types store multiple values in an ordered way. Types of Sequence Data Types List Tuple String Range Indexing Access elements using position (index starts from 0) my_list = [10, 20, 30] print(my_list[0]) # Output: 10 Negative indexing: print(my_list[-1]) # Output: 30 Slicing Extract a part of sequence: my_list = [10, 20, 30, 40, 50] print(my_list[1:4]) # Output: [20, 30, 40] Works similarly for list, tuple, and string my_tuple = (1, 2, 3, 4) print(my_tuple[1:3]) # (2,

3.

text = "Python" print(text[0:4]) # "Pyth" 2. What is the difference between a list and a tuple in Python? Why would you prefer a tuple over a list in real-world applications? feature List Tuple Mutability Mutable (can change) Immutable (cannot change) Syntax [ ] ( ) Performance Slower Faster Memory More memory Less memory Why prefer Tuple? Data should not change (e.g., coordinates, fixed values) Faster execution Safer (no accidental modification) Example: coordinates = (10, 20) # should not change 3. How are Python arrays different from lists? When should you use the array module instead of a list? List Can store different data types my_list = [1, "hello", 3.5] Array (from array module) Stores same data type only import array arr = array.array('i', [1, 2, 3]) Key Differences Feature List Array Data Types Mixed Same type only Speed Slower Faster (numeric) Memory More Less When to use array? Large numerical data Memory optimization needed Mathematical operations 4. Explain how dictionaries work internally in Python. Why are dictionary lookups faster compared to lists or tuples? Python dictionaries use a hash table Each key is converted into a hash value This hash decides where the value is stored Example my_dict = {"name": "Riya", "age": 20} print(my_dict["name"]) # Riya Why are lookups fast? Direct access using hash → O(1) time complexity No need to search entire data (unlike list) In lists: searching takes O(n) In dictionaries: direct access takes O(1) 5. What is the difference between sets and lists in Python? How do sets handle duplicate values, and where are sets useful in practical scenarios? Feature List Set Order Ordered Unordered Duplicates Allowed Not allowed Syntax [ ] { } How sets handle duplicates my_set = {1, 2, 2, 3} print(my_set) # Output: {1, 2, 3} Automatically removes duplicates Where sets are useful Removing duplicates Membership testing (fast) Mathematical operations (union, intersection) Example: a = {1, 2, 3} b = {3, 4, 5} print(a & b) # Intersection → {3} print(a | b) # Union → {1,2,3,4,5}

```
# String
s = "Python Programming"
print("Original String:", s)
print("Uppercase:", s.upper())
print("Lowercase:", s.lower())
print("Length:", len(s))
print("Slice:", s[0:6])
# List
list1 = [10, 20, 30, 40]
list1.append(50)
print("List after append:", list1)
list1.remove(20)
```

```
print("List after remove:", list1)
# Array
import array as arr
a = arr.array('i', [1,2,3,4])
a.append(5)
print("Array:", a)
# Tuple
t = (10,20,30)
print("Tuple:", t)
# Dictionary
d = {"Name":"Riya", "Age":20, "City":"Pune"}
print("Dictionary:", d)
print("Name:", d["Name"])
# Set
s = {1,2,3,4}
s.add(5)
print("Set:", s)
# Range
for i in range(1,6):
    print("Range value:", i)
```

```
Original String: Python Programming
Uppercase: PYTHON PROGRAMMING
Lowercase: python programming
Length: 18
Slice: Python
List after append: [10, 20, 30, 40, 50]
List after remove: [10, 30, 40, 50]
Array: array('i', [1, 2, 3, 4, 5])
Tuple: (10, 20, 30)
Dictionary: {'Name': 'Riya', 'Age': 20, 'City': 'Pune'}
Name: Riya
Set: {1, 2, 3, 4, 5}
Range value: 1
Range value: 2
Range value: 3
Range value: 4
Range value: 5
```

Conclusion 4/12/26, 10:14 PM Untitled11.ipynb - Colab Thus, we studied different sequence data types in Python such as String, List, Array, Tuple, Dictionary, Set, and Range. We also performed basic operations like slicing, append, remove, indexing, and iteration. These data types help in storing and managing data efficiently in Python programs

```
Start coding or generate with AI.
```